

MicroServices

Study Guide

Module 5

Developing BookStore –V.1.3
(Using Feign)

Author
Srinivas Dande





CourseCube®
(Formerly Java Learning Center)



Developing BookStore V-1.3

Applying Feign in our BookStore App Eco-System

- **Spring Cloud OpenFeign** – a declarative REST client for Spring Boot apps.
- You can use Feign as declarative Approach for Invoing Rest API
- Feign makes writing web service clients easier with pluggable annotation support,
- You can write the following code to invoke the Rest API using RestTemplate.

//Invoke BookPriceMS API using RestTemplate

```
RestTemplate bookPriceRest=new RestTemplate();  
String priceEndpoint="http://localhost:8000/offeredPrice/"+bookId;  
double offerPrice=bookPriceRest.getForObject(priceEndpoint, Double.class);
```

- You can simplify the Rest API calls with Feign as follows.

A) Write BookPriceProxy with BookPriceMS endponits.

```
@FeignClient(name="mybookPrice",url = "http://localhost:9000")  
public interface BookPriceProxy {  
  
    @GetMapping("/offeredPrice/{bookId}")  
    public double getOfferedPrice(@PathVariable Integer bookId);  
  
}
```

B) Inject the BookPriceProxy where ever you want to Rest API

```
@Autowired  
BookPriceProxy bookPriceProxy;
```

C) Invoking BookPrice-MS using RestTemplate (Remove this)

```
BookPriceInfo bpInfo=myBookPriceProxy.getBookPrice(bookId);
```



- Feign , Eureka and Ribbon are provided by Netflix
- Netflix Feign - provided by Netflix
- Spring Cloud has integrated Netflix Feign in the Initial days.
- When Netflix has stoped using Feign in their internal Projects , Nextflix has Provided – **Open Feign**
- **Spring Cloud** has integrated **Open Feign** now.

Spring Cloud Vesion:

Spring Cloud Release	Boot Version
2020.0.x aka Ilford	2.4.x, 2.5.x (Starting with 2020.0.3)
Hoxton	2.2.x, 2.3.x (Starting with SR5)
Greenwich	2.1.x
Finchley	2.0.x
Edgware	1.5.x
Dalston	1.5.x

Spring Cloud Tools :

Distributed/versioned configuration

- ✓ Spring Cloud Config
- ✓ Spring Cloud Bus

Service registration and discovery

- ✓ Eureka

Service-to-service calls

- ✓ OpenFeign

Load balancing

- ✓ Ribbon
- ✓ Spring Cloud Load Balancer

Circuit Breakers

- ✓ Hystrix



Step A : Using Feign in BookPriceMS-V1.3

- **BookPriceMS** is not calling any REST API of MicroServices.
- So No need to provide **Feign** Libraries for **BookPriceMS**.

Step B : Using Feign in BookSearchMS-V1.3

- **BookSearchMS** is calling the Rest Endponits of **BookPriceMS**
- So we need to provide **Feign** with **BookSearchMS**.

Working Steps:

1) Add the following dependency to pom.xml

```
<properties>
  <java.version>1.8</java.version>
  <spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.5.6</version>
</parent>

<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
  </dependency>
</dependencies>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```



2) Add @EnableFeignClients to Boot main class

```
@SpringBootApplication
@EnableSwagger2
@EnableFeignClients
public class BookSearchApplication {

}
```

3) Write BookPriceProxy with BookPriceMS endpoints

```
@FeignClient(name="mybookPrice",url = "http://localhost:9000")
public interface BookPriceProxy {

    @GetMapping("/bookPrice/{bookId}")
    public BookPrice getBookPrice(@PathVariable Integer bookId);

    @GetMapping("/offeredPrice/{bookId}")
    public double getOfferedPrice(@PathVariable Integer bookId);

}
```

4) Update BookServiceImpl.java.

```
//Invoking BookPrice-MS using RestTemplate (Remove this)

RestTemplate bookPriceRest=new RestTemplate();
String endpoint="http://localhost:9000/bookPrice/"+bookId;
BookPriceInfo bpInfo=bookPriceRest.getForObject(endpoint, BookPriceInfo.class);

@Autowired
MyBookPriceProxy myBookPriceProxy; (Add this)

//Invoking BookPrice-MS using Feign (Add this)
BookPriceInfo bpInfo=myBookPriceProxy.getBookPrice(bookId);
```



BookSearchMS – V-1.3: Files required

1. BookController.java	Same as V1.2
2. BookPriceInfo.java	Same as V1.2
3. BookInfo.java	Same as V1.2
4. BookInventoryInfo.java	Same as V1.2
5. BookRatingInfo.java	Same as V1.2
6. BookPriceProxy.java	Newly Added in V1.3
7. BookService.java	Same as V1.2
8. BookServiceImpl.java	Updated in V1.3
9. BookDAO.java	Same as V1.2
10. BookRatingDAO.java	Same as V1.2
11. BookInventoryDAO.java	Same as V1.2
12. Book.java	Same as V1.2
13. BookRating.java	Same as V1.2
14. BookInventory.java	Same as V1.2
15. JLCBookSearchConfig.java	Same as V1.2
16. BookSearchApplication.java	Updated in V1.3
17. application.properties	Same as V1.2
18. pom.xml	Updated in V1.3

6.BookPriceProxy.java

```
package com.jlcindia.feign;
```

```
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import com.jlcindia.booksearch.BookPriceInfo;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@FeignClient(name="MyBookPriceMS",url = "http://localhost:9000")
public interface BookPriceProxy {

    @GetMapping("/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId);

    @GetMapping("/offeredPrice/{bookId}")
    public double getOfferedPrice(@PathVariable Integer bookId);
}
```



8. BookServiceImpl.java

```
package com.jlcindia.booksearch;

import java.util.ArrayList;

import java.util.List;
import java.util.Optional;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.jlcindia.feign.BookPriceProxy;

import com.jlcindia.rabbitmq.BookInventoryInfo;
import com.jlcindia.rabbitmq.BookRatingInfo;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Service
@Transactional
public class BookServiceImpl implements BookService {

    static Logger log = LoggerFactory.getLogger(BookServiceImpl.class);

    @Autowired
    BookInventoryDAO bookInventoryDAO;

    @Autowired
    BookRatingDAO bookRatingDAO;

    @Autowired
    BookDAO bookDAO;

    @Autowired
    BookPriceProxy bookPriceProxy;
```



```
@Override
public List<Book> getBooks(String author, String category) {
    List<Book> mybooks = new ArrayList<>();

    if (author.equals("All Authors") && category.equals("All Categories")) {
        mybooks = bookDAO.findAll();
    } else if (author.equals("All Authors") && !category.equals("All Categories")) {
        mybooks = bookDAO.getBooksByCategory(category);
    } else if (!author.equals("All Authors") && category.equals("All Categories")) {
        mybooks = bookDAO.getBooksByAuthor(author);
    } else {
        mybooks = bookDAO.getBooksByAuthorAndCategory(author, category);
    }
    return mybooks;
}

@Override
public BookInfo getBookInfo(Integer bookId) {
    BookInfo bookInfo = new BookInfo();

    // 1. Book Details
    Optional<Book> myOpts = bookDAO.findById(bookId);
    if (myOpts.isPresent()) {
        Book book = myOpts.get();

        bookInfo.setBookId(book.getBookId());
        bookInfo.setBookName(book.getBookName());
        bookInfo.setAuthor(book.getAuthor());
        bookInfo.setPublications(book.getPublications());
        bookInfo.setCategory(book.getCategory());
    }

    // 2. Book Price Details
    //Invoking BookPrice Rest API with Fiegn
    BookPriceInfo bpInfo = bookPriceProxy.getBookPrice(bookId);

    bookInfo.setPrice(bpInfo.getPrice());
    bookInfo.setOffer(bpInfo.getOffer());

    // 3. Book rating Details
    BookRating bookRating = bookRatingDAO.getBookRatingByBookId(bookId);

    bookInfo.setAvgRating(bookRating.getAvgRating());
    bookInfo.setNumberOfSearches(bookRating.getNumberOfSearches());
}
```



```
return bookInfo;  
}
```

```
@RabbitListener(queues = "mybook.ratings.queue")
```

```
public void updateBookRating(BookRatingInfo bookRatingInfo) {  
    System.out.println("BookServiceImpl - updateBookRating");
```

```
    BookRating bookRating= new BookRating();
```

```
    bookRating.setBookId(bookRatingInfo.getBookId());
```

```
    bookRating.setAvgRating(bookRatingInfo.getAvgRating());
```

```
    bookRating.setNumberOfSearches(bookRatingInfo.getNumberOfSearches());
```

```
    bookRatingDAO.save(bookRating);
```

```
}
```

```
@RabbitListener(queues = "myinventory.queue")
```

```
public void updateBookInventory(BookInventoryInfo bookInventoryInfo) {  
    System.out.println("BookServiceImpl - updateBookInventory");
```

```
    BookInventory bookInventory= new BookInventory();
```

```
    bookInventory.setBookId(bookInventoryInfo.getBookId());
```

```
    bookInventory.setBooksAvailable(bookInventoryInfo.getBooksAvailable());
```

```
    bookInventoryDAO.save(bookInventory);
```

```
}
```

```
}
```

16.BookSearchApplication.java

```
package com.jlcindia.booksearch;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import org.springframework.cloud.openfeign.EnableFeignClients;
```

```
import springfox.documentation.swagger2.annotations.EnableSwagger2;
```

```
/*
```

```
 * @Author : Srinivas Dande
```

```
 * @company : Java Learning Center
```

```
 */
```



```
@SpringBootApplication
@EnableSwagger2
@EnableFeignClients
public class BookSearchApplication {
    static Logger log = LoggerFactory.getLogger(BookSearchApplication.class);

    public static void main(String args[]) {
        log.info(" BookSearchMS - Begin ");
        SpringApplication.run(BookSearchApplication.class, args);
        log.info(" BookSearchMS - End ");
    }
}
```

18. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.booksearch</groupId>
<artifactId>BookSearchMS</artifactId>
<version>1.3</version>
<packaging>jar</packaging>

<name>MyBookSearchMS</name>
<url>https://www.jlcindia.com</url>
<description>This is BookSeach MS developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.4.3</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<finalName>MyBookSearchMS</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

MicroService Name	Rest Endpoints
MyBookSearchMS	http://localhost:8000/mybooks/{author}/{category}
MyBookSearchMS	http://localhost:8000/mybook/{bookId}



Step C : Using Feign in PlaceOrderMS-V.1.3

- **PlaceOrderMS** is not calling any Rest API from other MicroServices.
- So **No** need to provide **Feign** Libraries for **PlaceOrderMS**.

Step D : Using Feign in UserRatingMS-V.1.3

- **UserRatingMS** is not calling any Rest API from other MicroServices.
- So **No** need to provide **Feign** Libraries for **UserRatingMS**.

Step E : Using Feign in BookStoreWeb-V1.3

- BookStoreWeb is calling the Rest Endponits of following MS
BookPriceMS (1) => BookPriceProxy
BookSearchMS (2) => BookSearchProxy
PlaceOrderMS (1) => PlaceOrderProxy
UserRatingMS (1) => UserRatingProxy

Working Steps:

1) Add the following dependency to pom.xml

```
<properties>
  <java.version>1.8</java.version>
  <spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.5.6</version>
</parent>

<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
  </dependency>
</dependencies>
```



```
<dependencyManagement>
<dependencies>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-dependencies</artifactId>
  <version>${spring-cloud.version}</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
```

2) Add @EnableFeignClients to Boot main class

```
@SpringBootApplication
@EnableSwagger2
@EnableFeignClients
public class BookStoreWebApplication{ ..... }
```

3) Write BookPriceProxy with BookPriceMS endpoints

```
@FeignClient(name="mybookPrice",url = "http://localhost:9000")
public interface BookPriceProxy {

  @GetMapping("/bookPrice/{bookId}")
  public BookPrice getBookPrice(@PathVariable Integer bookId);

  @GetMapping("/offeredPrice/{bookId}")
  public double getOfferedPrice(@PathVariable Integer bookId);

}
```

4) Write BookSearchProxy with BookSearchMS endpoints

```
@FeignClient(name="mybooksearch",url = "http://localhost:8000")
public interface BookSearchProxy {

  @GetMapping("/mybooks/{author}/{category}")
  public List<Book> getBooks(@PathVariable String author, @PathVariable String category);

  @GetMapping("/mybook/{bookId}")
  public BookInfo getBookById(@PathVariable Integer bookId);

}
```

5) Write PlaceOrderProxy with PlaceOrderMS endpoints

```
@FeignClient(name="myplaceorder", url = "http://localhost:7000")
public interface PlaceOrderProxy {

    @GetMapping("/myorders/{userId}")
    public List<Order> getOrdersByUserId(@PathVariable String userId);

}
```

6) Write UserRatingProxy with UserRatingMS endpoints

```
@FeignClient(name="myuserrating", url = "http://localhost:6500")
public interface UserRatingProxy {

    @GetMapping("/userRatings/{userId}")
    public List<UserRating> getUserRatingByUserId(@PathVariable String userId);

}
```

7) Update BookStoreServiceImpl.java.

A) Invoking BookPriceMS Rest API

```
@Autowired
BookPriceProxy bookPriceProxy;

// Invoke BookPriceMS Rest API with Feign
double offerPrice = bookPriceProxy.getOfferedPrice(bookId);
```

B) Invoking BookSearchMS Rest API

```
@Autowired
BookSearchProxy bookSearchProxy;

//Invoking BookSearch Rest API with Feign
List<Book> bookList = bookSearchProxy.getBooks(author, category);

//Invoking BookSearch Rest API with Feign
BookInfo bookInfo = bookSearchProxy.getBookById(bookId);
```



C) Invoking PlaceOrderMS Rest API

```
@Autowired
PlaceOrderProxy placeOrderProxy;

// Invoke PlaceOrderMS Rest API with Feign
List<Order> myorders = placeOrderProxy.getOrdersByUserId(userId);
```

D) Invoking UserRatingMS Rest API

```
@Autowired
UserRatingProxy userRatingProxy;

// Invoke UserRatingMS Rest API with Feign
List<UserRating> ratingsList = userRatingProxy.getUserRatingByUserId(userId);
```

BookStoreWeb – V-1.3: Files required

1. index.jsp	Same as V1.2
2. myheader.jsp	Same as V1.2
3. myfooter.jsp	Same as V1.2
4. showBooksList.jsp	Same as V1.2
5. showBookInfo.jsp	Same as V1.2
6. showCart.jsp	Same as V1.2
7. orderSuccess.jsp	Same as V1.2
8. addRating.jsp	Same as V1.2
9. ratingSuccess.jsp	Same as V1.2
10. ratingsList.jsp	Same as V1.2
11. bookstore.css	Same as V1.2
12. Book.java	Same as V1.2
13. BookInfo.java	Same as V1.2
14. Order.java	Same as V1.2
15. OrderItem.java	Same as V1.2
16. UserRating.java	Same as V1.2
17. BookStoreController.java	Same as V1.2
18. OrderFullInfo.java	Same as V1.2
19. OrderInfo.java	Same as V1.2
20. OrderItemInfo.java	Same as V1.2



21. UserRatingInfo.java	Same as V1.2
22. BookPriceInfo.java	Same as V1.2
23. BookPriceProxy.java	Newly Added in V1.3
24. BookSearchProxy.java	Newly Added in V1.3
25. PlaceOrderProxy.java	Newly Added in V1.3
26. UserRatingProxy.java	Newly Added in V1.3
27. BookStoreService.java	Same as V1.2
28. BookStoreServiceImpl.java	Updated in V1.3
29. JLCBookStoreWebConfig.java	Same as V1.2
30. BookStoreWebApplication.java	Updated in V1.3
31. application.properties	Same as V1.2
32. pom.xml	Updated in V1.3

23. BookPriceProxy.java

```
package com.jlcindia.feign;
```

```
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import com.jlcindia.bookstoreweb.BookPriceInfo;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@FeignClient(name="mybookprice",url = "http://localhost:9000")
public interface BookPriceProxy {

    @GetMapping("/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId);

    @GetMapping("/offeredPrice/{bookId}")
    public double getOfferedPrice(@PathVariable Integer bookId);
}
```

24. BookSearchProxy.java

```
package com.jlcindia.feign;
```

```
import java.util.List;
```

```
import org.springframework.cloud.openfeign.FeignClient;
```



```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

import com.jlcindia.bookstoreweb.Book;
import com.jlcindia.bookstoreweb.BookInfo;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@FeignClient(name="mybooksearch",url = "http://localhost:8000")
public interface BookSearchProxy {

    @GetMapping("/mybooks/{author}/{category}")
    public List<Book> getBooks(@PathVariable String author, @PathVariable String
category);

    @GetMapping("/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId);
}
```

25. PlaceOrderProxy.java

```
package com.jlcindia.feign;

import java.util.List;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

import com.jlcindia.bookstoreweb.Order;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@FeignClient(name="myplaceorder", url = "http://localhost:7000")
public interface PlaceOrderProxy {

    @GetMapping("/myorders/{userId}")
    public List<Order> getOrdersById(@PathVariable String userId);
}
```



26. UserRatingProxy.java

```
package com.jlcindia.feign;

import java.util.List;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

import com.jlcindia.bookstoreweb.UserRating;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@FeignClient(name="myuserrating", url = "http://localhost:6500")
public interface UserRatingProxy {

    @GetMapping("/userRatings/{userId}")
    public List<UserRating> getUserRatingByUserId(@PathVariable String userId) ;

}
```

28. BookStoreServiceImpl.java

```
package com.jlcindia.bookstoreweb;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.jlcindia.feign.BookPriceProxy;
import com.jlcindia.feign.BookSearchProxy;
import com.jlcindia.feign.PlaceOrderProxy;
```



```
import com.jlcindia.feign.UserRatingProxy;

import com.jlcindia.rabbitmq.OrderFullInfo;
import com.jlcindia.rabbitmq.OrderInfo;
import com.jlcindia.rabbitmq.OrderItemInfo;
import com.jlcindia.rabbitmq.UserRatingInfo;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Service
public class BookStoreServiceImpl implements BookStoreService {

    static Logger log = LoggerFactory.getLogger(BookStoreServiceImpl.class);

    Map<Integer, Book> booksMap = new LinkedHashMap<>();

    @Autowired
    RabbitTemplate rabbitTemplate;

    @Autowired
    BookPriceProxy bookPriceProxy;

    @Autowired
    BookSearchProxy bookSearchProxy;

    @Autowired
    PlaceOrderProxy placeOrderProxy;

    @Autowired
    UserRatingProxy userRatingProxy;

    public List<String> getAuthorsList() {
        List<String> authorsList = new ArrayList<>();
        authorsList.add("All Authors");
        authorsList.add("Srinivas");
        authorsList.add("Vas");
        authorsList.add("Sri");
        return authorsList;
    }
}
```



```
public List<String> getCategoryList() {
    List<String> catList = new ArrayList<>();
    catList.add("All Categories");
    catList.add("Web");
    catList.add("Spring");
    return catList;
}

public List<Book> getMyBooks(String author, String category) {
    System.out.println("BookStoreServiceImpl - getBooks()");

    if (author == null || author.length() == 0) {
        author = "All Authors";
    }
    if (category == null || category.length() == 0) {
        category = "All Categories";
    }

    //Invoking BookSearch Rest API with Feign
    List<Book> bookList = bookSearchProxy.getBooks(author, category);

    return bookList;
}

public BookInfo getBookInfoByBookId(Integer bookId) {
    System.out.println("BookStoreServiceImpl - getBookInfoByBookId()");

    //Invoking BookSearch Rest API with Feign
    BookInfo bookInfo = bookSearchProxy.getBookById(bookId);

    return bookInfo;
}

public Book getBookByBookId(Integer bookId) {
    System.out.println("BookStoreServiceImpl - getBookByBookId()");
    System.out.println(bookId);
    Book mybook = booksMap.get(bookId);

    return mybook;
}
```



```
public void placeOrder(Map<Integer, Book> mycartMap) {
// New Code
System.out.println("---2.BookStoreServiceImpl--placeOrder()----");
List<OrderItemInfo> itemList = new ArrayList<>();
double totalPrice = 0.0;
int totalQuantity = 0;

for (Book mybook : mycartMap.values()) {
Integer bookId = mybook.getBookId();

// Invoke BookPriceMS Rest API with Feign
double offerPrice = bookPriceProxy.getOfferedPrice(bookId);

OrderItemInfo item = new OrderItemInfo(0, bookId, 1, offerPrice);
itemList.add(item);

totalPrice = totalPrice + offerPrice;
totalQuantity = totalQuantity + 1;
}

Date today = Calendar.getInstance().getTime();
SimpleDateFormat formatter = new SimpleDateFormat("dd-MM-yyyy hh:mm");
String orderDate = formatter.format(today);
System.out.println(orderDate);

OrderInfo orderInfo = new OrderInfo(orderDate, "U-111", totalQuantity, totalPrice, "New");

OrderFullInfo orderFullInfo = new OrderFullInfo();
orderFullInfo.setOrder(orderInfo);
orderFullInfo.setItemsList(itemList);

// Sending Order Message to RabbitMQ
rabbitTemplate.convertAndSend("myorder.exchange","myorder.key",
orderFullInfo);
System.out.println("Order Placed");
}

@Override
public List<Order> getMyOrders(String userId) {

// Invoke PlaceOrderMS Rest API with Feign
List<Order> myorders = placeOrderProxy.getOrdersByUserId(userId);

return myorders;
}
```



```
}

public void addUserRating(UserRating userRating) {
    System.out.println("---2.BookStoreServiceImpl--addUserRating()----");

    //Sending UserRating Message RabbitMQ
    UserRatingInfo userRatingInfo=new UserRatingInfo(userRating.getUserId(),
    userRating.getBookId(), userRating.getRating(), userRating.getReview());
    rabbitTemplate.convertAndSend("myuser.ratings.exchange","myuser.ratings.key",
    userRatingInfo);

    System.out.println("Rating Added");
}

public List<UserRating> getMyRatings(String userId) {

    // Invoke UserRatingMS Rest API with Feign
    List<UserRating> ratingsList = userRatingProxy.getUserRatingByUserId(userId);

    return ratingsList;
}
}
```

30. BookStoreWebApplication.java

```
package com.jlcindia.bookstoreweb;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.openfeign.EnableFeignClients;

@SpringBootApplication
@EnableFeignClients
public class BookStoreWebApplication {
    static Logger log=LoggerFactory.getLogger(BookStoreWebApplication.class);

    public static void main(String args[]) {
        log.info(" BookStoreWeb - Begin ");
        SpringApplication.run(BookStoreWebApplication.class, args);
        log.info(" BookStoreWeb - End ");
    }
}
```



32.pom.xml

```
<project .....">

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookstoreweb</groupId>
<artifactId>BookStoreWeb</artifactId>
<version>1.3</version>
<packaging>jar</packaging>

<name>MyBookStoreWeb</name>
<url>https://www.jlcindia.com</url>
<description>This is BookStoreWeb developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.4.3</version>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<dependency>
<groupId>jstl</groupId>
<artifactId>jstl</artifactId>
<version>1.2</version>
</dependency>

<dependency>
<groupId>>taglibs</groupId>
<artifactId>standard</artifactId>
<version>1.1.2</version>
</dependency>

<dependency>
<groupId>org.webjars</groupId>
<artifactId>bootstrap</artifactId>
<version>4.3.1</version>
</dependency>

<dependency>
<groupId>org.webjars</groupId>
<artifactId>jquery</artifactId>
<version>3.4.1</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>
```



```
</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<finalName>MyBookStoreWeb</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

MicroService Name	Rest Endpoints	Used By
MyBookPriceMS	http://localhost:9000/bookPrice/{bookId}	MyBookSearchMS
MyBookPriceMS	http://localhost:9000/offeredPrice/{bookId}	MyBookStoreWeb
MyBookSearchMS	http://localhost:8000/mybooks/{author}/{category}	MyBookStoreWeb
MyBookSearchMS	http://localhost:8000/mybook/{bookId}	MyBookStoreWeb
MyPlaceOrderMS	http://localhost:7000/myorders/{userId}	MyBookStoreWeb
MyUserRatingMS	http://localhost:6500/userRatings/{userId}	MyBookStoreWeb